Lab 2：Bomb Lab 实验报告

1. Phase 1：多步数据流组合

1.1 关键汇编

```
; step0
lea eax, [rsi+0x8b9d]
xor eax, edi

; step1
lea eax, [rdi+rsi*4]
xor eax, 0x7cc4

; step2
lea eax, [rdi+rdi*4]
add eax, esi
xor eax, 0xf35c
```

lea eax,[rdi+rdi*4] 表示 (5× rdi)。p1_compute 把 challenge 的不同部分送入以上三个步骤，并将结果组合。

1.2 恢复的计算关系

```
M  = low32(C >> 16)
s0 = (H + 0x8b9d) ^ L
s1 = (H + 4*M) ^ 0x7cc4
s2 = (5*s1 + L) ^ 0xf35c
answer = s2 ^ s0
```

---

2. Phase 2：循环递推数列

2.1 关键汇编

第一个元素由 challenge 高 32 位中的 6 位产生：

```
shr rax, 0x20
and eax, 0x3f
add eax, 0x0c
```

后续五个元素在循环中生成：

```
shr rax, cl
and eax, 0x3f
add eax, edx
xor eax, esi
lea eax, [rax+rax*2]
```

其中 ecx 从 20 开始，每轮增加 4；edx 是下标 (i)；esi 保存前一项。

2.2 恢复的计算关系

```
a[0] = ((C >> 32) & 0x3f) + 12

a[i] = 3 * ((((C >> (16 + 4*i)) & 0x3f) + i) ^ a[i-1]),
       i = 1,...,5
```

2.3 推理结论

每轮先从 challenge 中取 6 位，加上下标，与前一项异或，再乘 3。最终按顺序输出：

```
a[0] a[1] a[2] a[3] a[4] a[5]
```

---

3. Phase 3：跳转表与多分支

3.1 分支参数

phase_03 先构造跳转表下标和传入分支函数的参数：

```
index = L & 7
x = low32((C >> 3) ^ (C >> 32))
```

关键间接跳转形式为：

```
jmp QWORD PTR [rax*8+0x405820]
```

因此不能只按照函数名的编号猜测分支，而要读取跳转表中每个表项的实际目标地址。

3.2 跳转表恢复结果

index	实际分支	answer
0	p3_case_7	5*x + 0x3cd93
1	p3_case_3	(x ^ 0x409f) + (x >> 1)
2	p3_case_1	5*x + 0x3395a
3	p3_case_6	5*x + 0xb422
4	p3_case_0	3*x + 0xc7a4
5	p3_case_4	(x ^ 0x16c8) + (x >> 1)
6	p3_case_2	(4*x) ^ 0xd3f3
7	p3_case_5	(4*x) ^ 0x37c8

3.3 推理与纠错

最初曾把 p3_case_5 中的 lea eax,[rdi*4] 错看成 (5x)，导致公开测试中 seed 99 和 123 失败。重新核对后确认它是 (4x)，修正后 Phase 3 的全部公开 seed 通过。这也说明跳转表必须逐项验证，不能只测试少数分支。

```
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: FAIL (wrong_answer)
Phase 3 seed 123: FAIL (wrong_answer)
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 58/60
```

---

4. Phase 4：递归二分路径

4.1 入口参数

```
target = L & 0x7f
key = low32(C >> 7) ^ H
answer = p4_walk(target, 0, 127, key)
```

4.2 关键汇编

中点计算：

```
mov edi, edx
sub edi, esi
shr edi, 1
add edi, esi
```

对应：

```
mid = low + ((high - low) >> 1)
```

4.3 恢复的递归关系

```
walk(target, low, high, key):
    mid = low + ((high-low) >> 1)

    if target == mid:
        return (mid + 0x43) ^ key

    mix = mid + key

    if target < mid:
        child = walk(target, low, mid-1, key)
        return ((5*child) ^ mix) ^ 0x04
    else:
        child = walk(target, mid+1, high, key)
        return ((5*child) ^ mix) ^ 0x1f
```

5. Phase 5：链表与滚动哈希

5.1 节点结构

通过节点指针和偏移访问模式可以恢复结构：

```
struct node {
    struct node *next;  // offset 0
    uint32_t value;     // offset 8
    uint32_t weight;    // offset 12
};
```

关键访存关系可概括为：

```
mov ..., QWORD PTR [node+0x0]   ; next
mov ..., DWORD PTR [node+0x8]   ; value
mov ..., DWORD PTR [node+0xc]   ; weight
```

5.2 初始参数与更新关系

```
start = L & 7
steps = ((L >> 3) & 3) + 1
hash = low32(C >> 6) ^ H ^ 0x4067
```

每访问一个节点：

```
hash = ((hash + node.weight) * 5) ^ node.value
```

5.3 恢复的节点数据

位置	value	weight	位置	value	weight
0	273	18063	6	2144	14539
1	1915	59068	7	91	4275
2	570	13961	8	2348	48741
3	784	51946	9	1571	34307
4	2661	17775	10	2910	24637
5	1387	50792			

八个头指针指向同一条链中的不同位置。根据 start 选取入口，遍历 steps 次并更新哈希，最后输出：

```
hash final_value
```

其中 final_value 为遍历结束时节点的 value。

---

6. Phase 6：函数指针的两次间接调用

6.1 参数构造

```
x  = low32(C >> 6) ^ H
i1 = L & 7
i2 = (L >> 3) & 7
```

6.2 关键汇编

```
call QWORD PTR [rax*8+0x405e00]
xor  eax, 0x1491
call QWORD PTR [rbx+0x405e00]
add  eax, 0xfff1
```

这表示先调用下标 i1 对应的函数，再异或常量，然后调用 i2 对应的函数，最后加常量。

6.3 函数表

下标	(F_i(x))
0	ROL32(x,3) ^ 0xaef9
1	((x >> 2) ^ x) + 0x6ffa
2	(4*x + 0x9e32) ^ (5*x)
3	ROR32(x,3)
4	(x ^ 0x9d4c) + (x << 7)
5	ROL32(x,3) ^ 0xaef9
6	((x >> 2) ^ x) + 0x6ffa
7	(x + 0x53f) ^ (x >> 2)

因此：

```
answer = F_i2(F_i1(x) ^ 0x1491) + 0xfff1
```

ROL32 和 ROR32 必须按 32 位循环移位实现，不能使用普通右移代替。

---

7. Honor：隐藏关入口与计算关系

7.2 整数参数

```
s  = low32(context) & 3
cf = low32(challenge) ^ high32(challenge)
xf = low32(context) ^ high32(context)
arg = cf ^ xf
```

secret_mix 的关键汇编：

```
mov eax, edi
xor edi, 0x12f
shr eax, 1
xor eax, 0xcfb2
add eax, edi
```

恢复为：

```
mixed  = ((arg >> 1) ^ 0xcfb2) + (arg ^ 0x12f)
second = mixed ^ (s * 0x12f) ^ low32(context >> 16)
third  = second ^ (xf + s + 0xcfb2 + (cf >> 1))
```

最终输出格式：

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN s second third
```

---

8. 验证记录

8.1 公开测试

最终结果：

- Phase 1-6：60/60 通过；
- Honor：公开 seed 全部通过。

Phase 3 修正前为 58/60；修正 p3_case_5 后恢复为 60/60。

8.2 公开列表以外的 seed

选择 seed=314159。该 seed 不在 tests/public.json 的公开列表中。

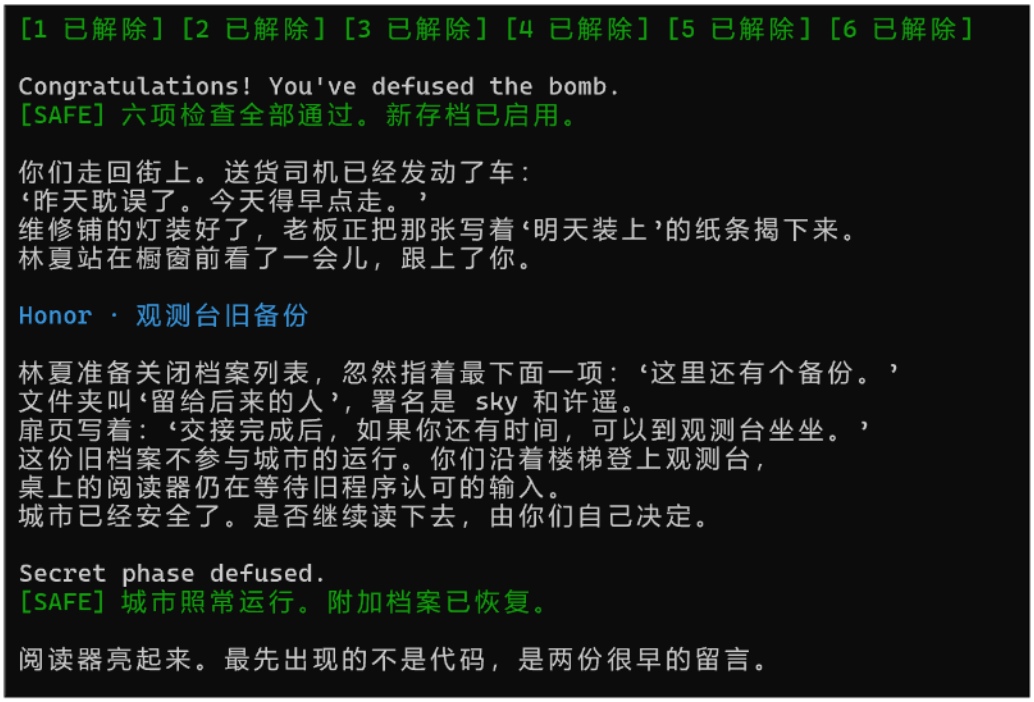
验证命令：

```
{ ./defuser --seed 314159; ./defuser --seed 314159 --secret; } \
    | ./bomb --seed 314159
```

求解器输出记录：

关卡	输出
Phase 1	2468425545
Phase 2	74 213 741 2172 6330 18846
Phase 3	5 1882793476
Phase 4	1551722502
Phase 5	2075369809 2661
Phase 6	2076900131
Honor	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN 2 1622926340 2605017684

运行结果中依次出现 Phase 1-6 已解除、最终通关提示以及 Secret phase defused.，说明普通六关和 Honor 均通过端到端验证。



最终 PDF 将直接嵌入该截图；当前 Markdown 审阅稿使用相对路径标示截图位置。

---

10. 工具与引用资料

10.1 工具用途

- make / GCC：使用课程规定的 C11 和警告选项编译 defuser.c。
- public_test.py：对多个公开 seed 自动验证 Phase 1-6 与 Honor。
- ChatGPT：整理实验报告和 Git 提交流程。

10.2 引用资料

1. 课程 Bomb Lab 实验页面：
https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/
2. GDB 官方文档：
https://sourceware.org/gdb/current/onlinedocs/gdb.html
3. GNU Binutils 官方文档：
https://sourceware.org/binutils/docs/